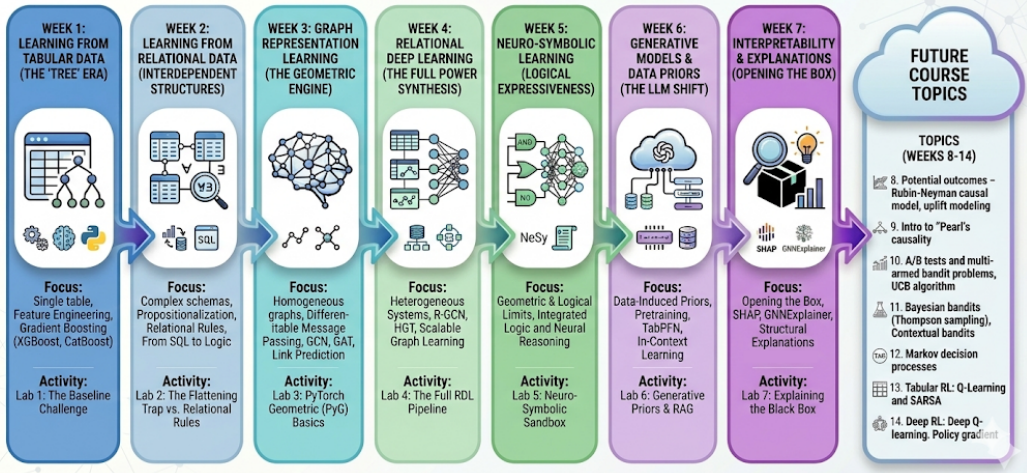


LEARNING FROM STRUCTURED DATA: A 7-WEEK JOURNEY

Narrative Arc: Increasing Structure → Increasing Inductive Bias → Increasing Reasoning Power



Course Map of Week 6: Learning with LLMs

- 1 The Paradigm Shift & Universal Learners
- 2 Tabular Foundation Models & ICL
- 3 Structured Input & Graphormer
- 4 Structured Output & Constrained Generation
- 5 Interacting with Structure (Text-to-SQL & GraphRAG)

The Paradigm Shift: Priors vs. Data

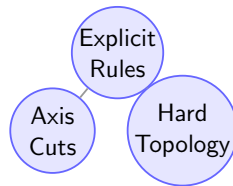
Weeks 1–5: Architectural Priors

- **Tabular (W1):** Trees force axis-aligned cuts.
- **Relational (W2):** Logic forces boolean strictness.
- **Graphs (W3-4):** GNNs enforce local aggregation.
- **Limit:** Data "physics" explicitly wired into the architecture.

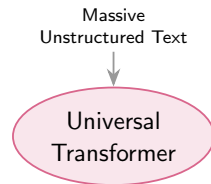
Week 6: Data-Induced Priors (The LLM Shift)

- **The Transformer:** Almost universal, "prior-free" neural architecture.
- **The Engine:** Learns structure via internet-scale pretraining.
- **Goal:** Can a universal sequence learner replace specialized structured models?

Architectural Priors



Data-Induced Priors

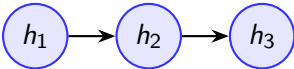


Why Move Beyond Recurrence?

The Recurrent Bottleneck (W4/W5 Context)

- RNNs/LSTMs process sequences strictly left-to-right.
- **Problem:** $\mathcal{O}(n)$ sequential operations $\rightarrow$ unparallelizable bottleneck.

Recurrent Model $\mathcal{O}(n)$ Path

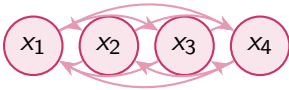


Sequential Bottleneck

The Transformer Solution

- Replaces recurrence entirely with **Self-Attention**.
- Max interaction distance between any two tokens drops to $\mathcal{O}(1)$.
- **Result:** Boundless parallelization across GPU hardware.

Self-Attention $\mathcal{O}(1)$ Path



Deconstructing Self-Attention (The Engine)

The "Fuzzy" Hash Table

- Treat each token as a **Query** searching for relevant **Keys**.
- Returns a weighted sum of **Values**.

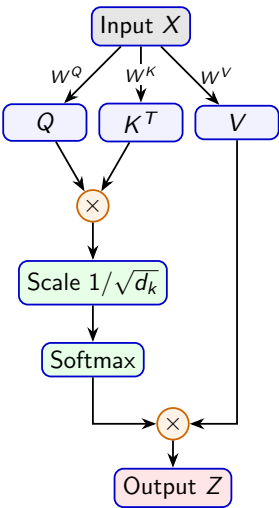
Linear Projections

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

Scaled Dot-Product Attention

$$\text{Attention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- *Scaling*: $\sqrt{d_k}$ stabilizes gradients for large dot products.



Transformers $\equiv$ Graph Neural Networks

The NLP View vs. The Graph View

- **NLP:** A sequence of ordered word tokens.
- **Graph:** A **fully-connected graph** (tokens = nodes).

Implicit Edge Learning

- GNNs rely on a hard Adjacency Matrix (Explicit edges).
- Transformers compute an $n \times n$ Attention Matrix (Soft, learned edges).

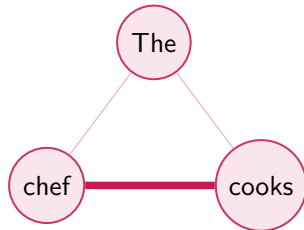
Message Passing Isomorphism

- Self-Attention *is* a Graph Attention Network on a complete graph.
- Connections are dictated by semantic similarity, not topology.

1D Sequence (NLP)



Fully-Connected Graph (GAT)



The Mathematical Isomorphism (GAT vs Transformer)

Graph Attention Network (Sparse)

$$h_i^{(k)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j^{(k-1)}$$

- Attention α_{ij} computed only for predefined neighborhood $\mathcal{N}(i)$.

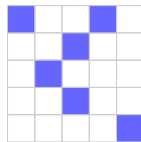
Transformer Self-Attention (Dense)

$$z_i = \sum_{j=1}^n \alpha_{ij} (x_j W^V)$$

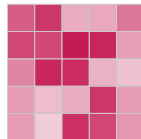
- Attention α_{ij} computed for *all* n tokens in the sequence.

The Core Difference: Transformers drop explicit structural priors ($\mathcal{N}(i)$) for dense $\mathcal{O}(n^2)$ global interactions.

Sparse Adjacency (GAT)



Dense Attention (Transformer)



The Hardware Lottery

If mathematically equivalent, why do Transformers dominate?

The answer lies in **hardware optimization**, not mathematics.

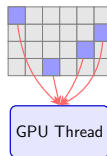
GNNs: Sparse Scatter-Gather

- Rely on COO Adjacency matrices → fragmented memory access.
- Result: Frequent GPU cache misses and slow processing.

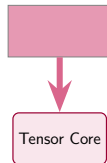
Transformers: Dense MatMul

- Dense $(n \times d) \times (d \times n)$ operations.
- Tensor Cores heavily optimize dense contiguous operations.
- **Result:** Faster to brute-force $\mathcal{O}(n^2)$ densely!

Sparse Access (GNN)



Dense Access (LLM)



The Symmetries Bottleneck: Permutation Invariance

Engagement Question

- *If we scramble word order (incl. Positional Encodings), does the output change?*

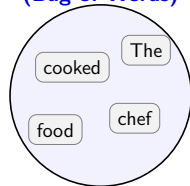
The Reality of Self-Attention

- **Answer: NO.** Self-Attention is fundamentally a **Set Operation**.
- The model treats language like an unordered "bag of words".

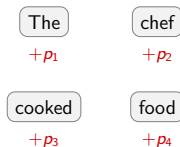
Injecting Sequence Priors

- Attention is topologically blind; we must force 1D structure.
- *Method:* Add positional vectors to embeddings ($\tilde{x}_i = x_i + p_i$).

1. Topologically Blind (Bag of Words)



2. Injecting 1D Prior



The Sequence-Structure Bottleneck

The Core Conflict of Week 6

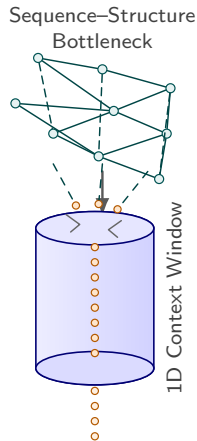
- **LLMs:** Inherently 1D sequential models (enforced by p_i).
- **Structured Data:** 2D (Tables), N-D (Graphs), Permutation-Invariant.

The Return of the "Flattening Trap"

- Naive approach: Serialize graphs/tables into 1D text.
- *Result:* Destroys topology, bloats context window, and reinvents Propositionalization failures.

Mapping native 2D/N-D structure

- *Next:* Tabular Foundation Models & Structural Encodings.



Return to Week 1: Can a Transformer Beat XGBoost?

The Week 1 Baseline (GBDTs)

- XGBoost natively matches jagged tabular manifolds via axis-aligned cuts.
- Limitation: Requires full SGD retraining per task.

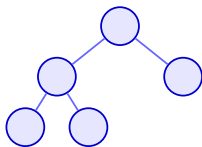
The Universal Sequence Learner

- Transformers learn priors via pre-training.
- *Hypothesis*: Can self-attention implicitly build decision boundaries?

The Serialization Trap

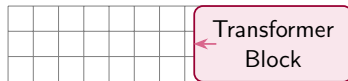
- Flattening a table into CSV destroys column independence and exhausts the context window.

Axis-Aligned
(XGBoost)



?

Universal
(Transformer)



The Native Approach: FT-Transformer

Feature Tokenizer Transformer

- Adapts Transformer directly for tabular rows.

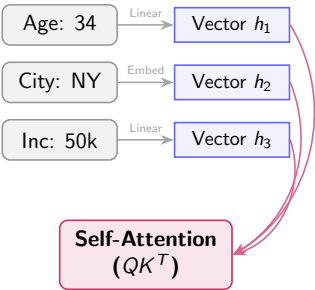
Columns as Tokens

- Each tabular *feature* becomes a separate token.
- Categorical:** Embedded via dictionaries.
- Numerical:** Embedded via linear projections.

Self-Attention on Features

- QK^T learns cross-column feature interactions natively.
- Drawback: Requires SGD training for *each* new dataset (Not a Foundation Model!).

Raw 2D Row:



The Magic of In-Context Learning (ICL)

What is In-Context Learning?

- Recognizing patterns and performing inference entirely within the context window.

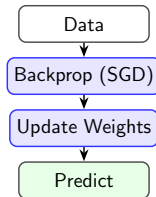
The Core Distinction

- **Standard ML:** Parameter updates via backprop.
- **ICL:** Requires **ZERO parameter updates**. Operates on pre-trained activations in one forward pass.

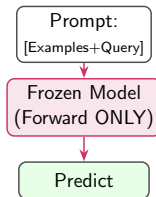
Engagement Question: *If weights are entirely frozen, how is the model "learning"?*

- **Answer:** It runs a dynamic inference algorithm learned during massive pre-training!

Flow 1: Standard ML



Flow 2: ICL



No Weight Updates!

ICL: Parametric vs. Working Memory

The Dual-Memory Hypothesis

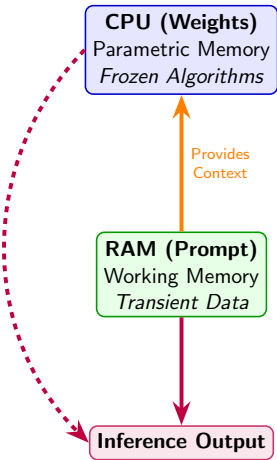
- How can a model reason over unseen data without updating weights?
- *Answer:* It leverages two distinct forms of memory.

1. Parametric Memory (Long-Term)

- The frozen matrices ($W^Q, W^K \dots$) from pre-training.
- Stores the **Algorithms** (how to compare, how to cluster).

2. Working Memory (Short-Term)

- The KV-Cache of the Context Window.
- Stores the **Data** (the specific task examples).
- **Result:** The frozen weights act as a CPU processing the prompt's RAM!



Mechanistic Interpretability: Induction Heads

Opening the Black Box

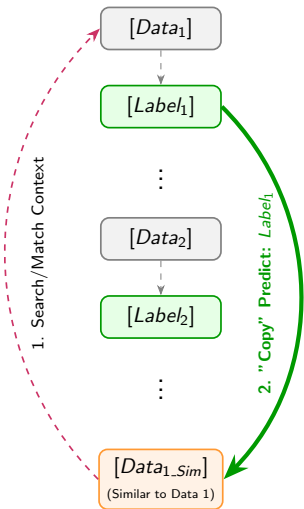
- How does the "CPU" actually execute ICL? We look to **Mechanistic Interpretability**.

The Induction Head Circuit

- Specific attention circuits responsible for "copy/paste" reasoning.
- Matches patterns: $[A][B] \dots [A] \rightarrow \text{Predict } [B]$.

In-Context Tabular Inference

- Induction Heads dynamically bind inputs (features) to outputs (labels).
- Acts as a dynamic nearest-neighbor algorithm entirely within frozen activations!



TabPFN: The Tabular Foundation Model

The Goal: A Universal Tabular Engine

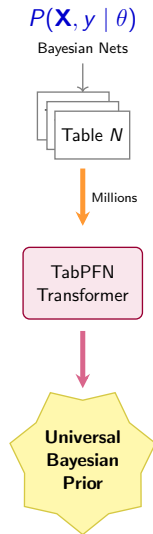
- Can we pre-train a model to solve *any* small tabular dataset instantly?

Prior-Data Fitting (PFN)

- TabPFN is trained on **millions of synthetic datasets**.
- Generated via Structural Causal Models (Bayes Nets).

Learning the Universal Prior

- The Transformer internalizes the mathematical rules of tabular data (the "Bayesian Prior").
- Expects jagged boundaries and noise natively.



Generating the Tabular Universe (SCMs)

The Cold-Start Problem

- We cannot scrape the internet for tabular data (due to privacy & corporate silos).
- We mathematically synthesize a "Tabular Universe".

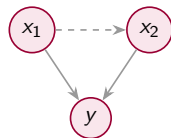
Structural Causal Models (SCMs)

- Randomly sample graph structures to get dependencies.
- Randomly assign functions (non/linear) between vars.

Hyperparameter Randomization

- Randomly vary the number of classes, feature scales, and noise distributions.
- **Result:** TabPFN sees millions of distinct topological manifolds before it ever sees a real dataset!

1. Random Topology



2. Random Functions

$$y = \sin(x_1) + x_2^2 + \epsilon$$

3. Sample Table

x1	x2	y
0.2	1.1	0
0.9	-0.1	1
...	...	.

Inference with TabPFN (Zero-Shot Tabular ML)

The Inference Paradigm Shift

- No hyperparameter tuning. No Cross-Validation. No SGD.

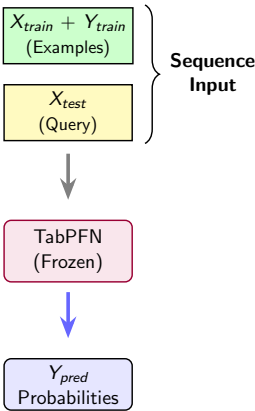
The "Prompt" Structure

- **Context:** Feed the entire Training Set into the window.
- **Query:** Append the Test Set at the sequence end.

A Single Forward Pass

- Induction heads map test features to the training prior.
- Outputs exact probabilities for Y_{test} instantaneously.

The Structural Prompt



The Scaling Profile: TabPFN vs. XGBoost

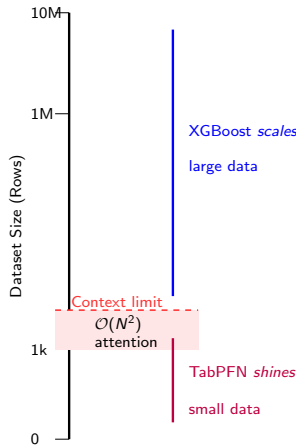
The Ultimate Trade-off: If TabPFN is instant, why does XGBoost still exist?

Algorithm Comparison

- **XGBoost:** Gradient Boost optimization, Tree Traversal inference. Scales to millions via $\mathcal{O}(N \log N)$.
- **TabPFN:** Pre-trained Prior optimization, In-Context inference. **Bounded by Context Limit!**

The Sequence-Structure Bottleneck

- $\mathcal{O}(N^2)$ global attention prevents massive tables.
- *Conclusion:* TabPFN dominates **small** datasets ($< 10k$ rows). XGBoost is king for **large** data.



The Graph Serialization Trap

The LLM Temptation

- *Goal:* Use LLMs to reason over graph structures.

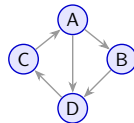
Naive Approach: Textual Propositionalization

- Flatten a graph neighborhood into a 1D string (JSON/MD).

The Return of Week 2's Flattening Trap

- **Destroys** topological geometry (cycles, cliques).
- Imposes arbitrary order on unordered sets.
- Rapidly exhausts the 1D context window (Context Bloat).

3D Graph



Serialization



1D Context
Window Limits

```
{ "n": "A",  
  "adj": ["B", "C"],  
  ... OVERFLOW }
```

Bloat!

Topological Blindness in Transformers

Engagement Question: *Is feeding a 5-hop graph to an LLM as a flat string true graph learning?*

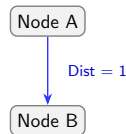
NLP Positional Encodings (1D)

- Language has a canonical 1D grid.
- Transformers use vectors (RoPE) to inject 1D distance.

The Graph Reality (Non-Euclidean)

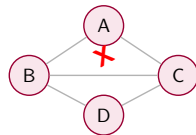
- Graphs lack a canonical grid.
- Standard 1D positional encodings are meaningless!
- **Result:** The Transformer is topologically blind.

1D Text Sequence



Clear 1D sequence index
 $i = 1 \rightarrow i = 2$

Non-Euclidean Graph



No starting node. No "left" or "right".
1D encodings fail.

Graphormer: Centrality & Edge Encodings

1. Centrality Encoding (Node Importance)

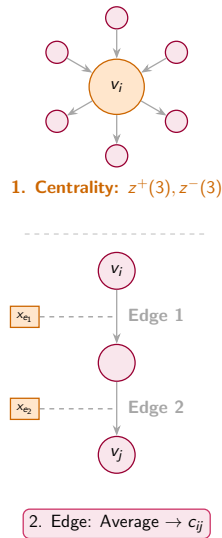
- Standard attention ignores that "Hub" nodes dictate network flow.
- *Fix:* Add learnable in/out-degree vectors directly to the input features: $h_i^{(0)} = x_i + z_{deg}^- + z_{deg}^+$.

2. Edge Encoding (Relational Features)

- Standard attention ignores the *journey* (edge features like bond types or timestamps).
- *Fix:* Compute the average of the edge features along the shortest path between nodes i and j .

Direct Attention Injection

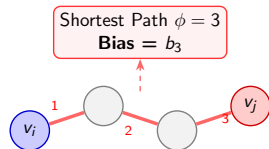
- This edge average (c_{ij}) is added directly to the attention score, explicitly evaluating path quality!



Spatial Bias & The Master Equation

3. Spatial Encoding (Topological Distance)

- Replaces 1D sequence position with **Shortest Path Distance** $\phi(v_i, v_j)$.
- Assigns a learnable scalar bias b_ϕ to each hop.



The Graphormer Master Equation

$$A_{ij} = \underbrace{\frac{(h_i W^Q)(h_j W^K)^T}{\sqrt{d}}}_{\text{Semantic}} + \underbrace{b_{\phi(v_i, v_j)}}_{\text{Spatial}} + \underbrace{c_{ij}}_{\text{Edge}}$$

Adaptive Structural Attention

- A_{ij} synthesizes content and structure.
- If b_ϕ decays with distance, attention naturally fades for distant nodes, mimicking a GNN globally

QK^T (Content)

+

$B_\phi + C$ (Structure)

=

A_{ij} (Graphormer)

Expressivity: Breaking the 1-WL Limit

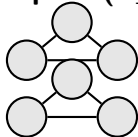
The Ultimate Expressivity Test

- Standard MPNNs are bounded by the 1-WL test (blind to cycles).

Graphormer $\geq$ Message Passing GNNs

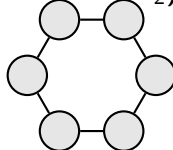
- Shortest-Path Distances (ϕ) distinguish some graphs that 1-WL considers identical!
- **Mathematical Note:** By properly injecting structural priors, Graphormer encompasses GCN, GIN, and GraphSAGE as special cases.

Graph 1 (G_1)



$SPD_{max} = 1$

Graph 2 (G_2)



$SPD_{max} = 3$

Standard GNN (1-WL)
 $Embed(G_1) = Embed(G_2)$

Graphormer
 $Embed(G_1) \neq Embed(G_2)$

The Output Bottleneck: Probabilistic vs. Deterministic

The Generative Reality (System 1)

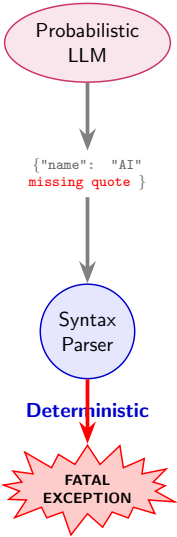
- Autoregressive LLMs predict tokens probabilistically.
- Inherently susceptible to syntax deviations.

The Structured Reality (System 2)

- Databases and Logic Engines are strictly deterministic.
- A single missing comma crashes the pipeline.

The Core Problem

- How do we interface a probabilistic generator with a deterministic executor?
- *Naive Solution:* Prompt Engineering → **Fails at scale.**



The Mathematics of Logit Masking

Standard Autoregressive Decoding

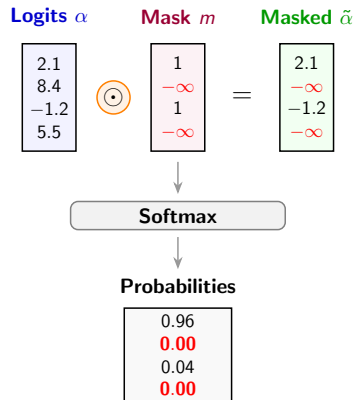
- LLM outputs logit vector $\alpha \in \mathbb{R}^N$ for vocab.
- $s_{t+1} \sim \text{Categorical}(\text{softmax}(\alpha))$

Constrained Decoding (Logit Masking)

- Define a boolean mask $m(S_t) \in \{-\infty, 1\}^N$.
- Mask the logits: $\tilde{\alpha} = m(S_t) \odot \alpha$

The Absolute Guarantee

- If token i violates schema,
 $m_i = -\infty \implies \text{logit } -\infty$.
- $\text{softmax}(-\infty) = 0$. Model **cannot** sample it!



Modeling Syntax via Finite State Machines (FSM)

Computing the mask $m(S_t)$

- Structured formats (JSON, SQL) compile to **Finite State Machines (FSMs)**.

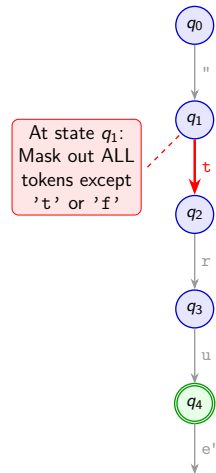
The FSM Tuple

- $M = (Q, \Sigma, \delta, q_0, F)$
- States (Q), Alphabet (Σ), Transitions (δ).

State-Driven Masking

- As LLM generates tokens, we traverse the FSM.
- Current state q_t defines valid next transitions.
- Invalid transitions $\implies$ Masked to $-\infty$.

JSON Boolean FSM



The Runtime Bottleneck & The Indexing Trick

The $\mathcal{O}(N)$ Runtime Problem

- Checking all 50k tokens against the FSM at every step causes severe latency.

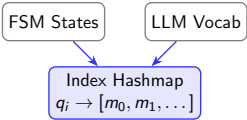
The Outlines Solution: Vocabulary Indexing

- *Offline*: Pre-compute which tokens are accepted by which FSM states.

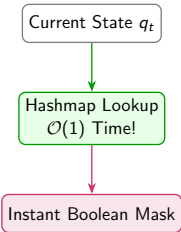
$\mathcal{O}(1)$ Mask Retrieval

- *Runtime*: Look up current FSM state in a hash map to get the mask instantly.

Step 1: Offline



Step 2: Runtime



High-Level Orchestration: Pydantic & Instructor

Beyond Syntax: Semantic Constraints

- FSMs guarantee *Syntax* ("Output an int").
- FSMs cannot guarantee *Semantics* ("Integer must be a valid DB ID").

The Instructor Framework

- Elevates constraints to Application layer via Python's Pydantic.

Validation Hooks

- Allows custom `@model_validator` logic.
- Can query the Database **during** generation to verify entities!

```
from pydantic import BaseModel
import database

class User(BaseModel):
    id: int
    name: str

    @model_validator(mode='after')
    def check_id(self):
        if self.id not in db.users:
            raise ValueError("ID Error!")
        return self
```

The AI Self-Correction Loop (Self-Healing)

Handling Validation Failures

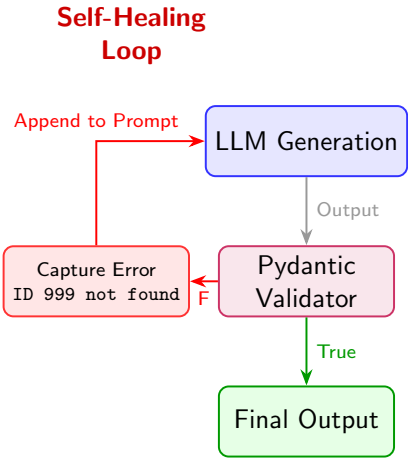
- What happens when the Pydantic validator fails?

The "Re-Ask" Paradigm

- Framework catches the `ValidationError`.
- Appends Python error back into the LLM's prompt.

Iterative Reasoning

- *Prompt:* "Output failed: ID 999 not found. Correct."
- Transforms static LLM to self-correcting engine.



Engagement Question: The Constraint Tradeoff

Engagement Question

- *If we mathematically force an LLM to output valid JSON, does semantic quality improve or degrade?*

The Hidden Cost of Strict Structure

- **Answer:** It can make the answer **worse!**
- LLMs rely on generating text to "think" (Chain-of-Thought).
- Immediate JSON constraints deprive reasoning space.
- *The Fix:* Allow a <thought_process> string field **before** the final answer field.

Strict JSON (Fails)

```
{  
  "answer": 42  
}
```



CoT JSON (Succeeds)

```
{  
  "reasoning": "20+...",  
  "answer": 45  
}
```



Summary: Bridging Sequence and Structure

The Generative Problem Solved

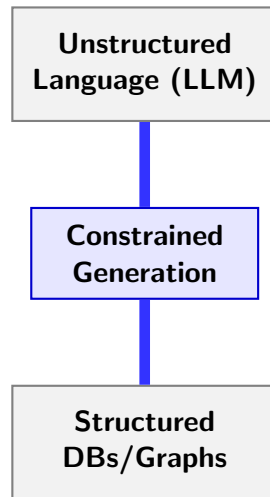
- FSM Logit Masking eliminates syntactic hallucinations ($\mathcal{O}(1)$ latency).

The Semantic Problem Solved

- Pydantic Validation & Self-Healing Loops eliminate logical hallucinations.

Setting the Stage for Module 5

- We possess a reliable, deterministic **Text-to-Structure** engine.
- *Next:* Translate Natural Language into SQL and GraphRAG.



The Grand Synthesis: System 1 Meets System 2

System 1: Generative LLMs

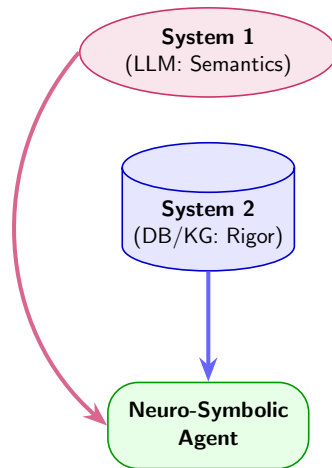
- **Strengths:** Broad semantic parsing.
- **Weaknesses:** Probabilistic, hallucinations.

System 2: Structured DBs

- **Strengths:** Deterministic, rigorous retrieval.
- **Weaknesses:** Rigid syntax requirements.

The Neuro-Symbolic Agent

- **Goal:** Use LLM to translate natural intent into formal logic; delegate execution to the database!



Semantic Parsing (Text-to-SQL)

What is Semantic Parsing?

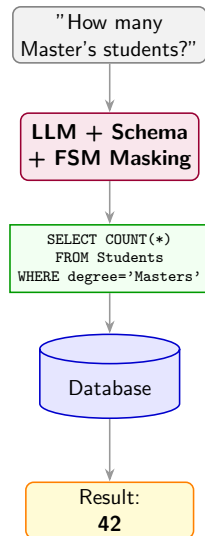
- Translating Natural Language (NL) into executable logic (SQL/Cypher).

The Inverse of Week 2

- *Week 2 (ILP)*: Data $\rightarrow$ Logical Rules.
- *Week 6*: Intent $\rightarrow$ Rules $\rightarrow$ Data.

The Core Pipeline

- 1 Feed DB Schema + NL Query to LLM.
- 2 Use Constrained Decoding to ensure syntax.
- 3 Execute on DB for grounded answers.



Enhancing Text-to-SQL (Structure-Guided Prompts)

Schema Hallucinations

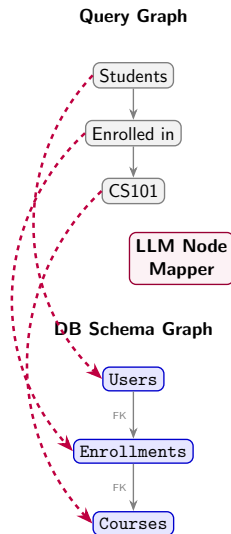
- LLMs often join incorrect tables on complex schemas.

Structure-Guided Generation

- Avoid flat text schemas! Parse natural language into a semantic **Query Graph**.

Sub-Task Execution

- LLM explicitly maps Query Graph nodes to Schema nodes.
- Enables iterative, topological sub-task resolution.



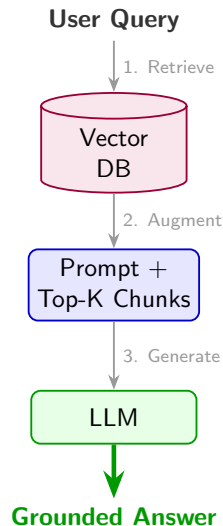
Bridging Text and Data: What is RAG?

The LLM Knowledge Problem

- LLMs have frozen weights (outdated information).
- Prone to hallucinations on proprietary enterprise data.

Retrieval-Augmented Generation (RAG)

- **1. Retrieve:** Search a Vector Database for text chunks relevant to the user's query.
- **2. Augment:** Paste those retrieved chunks directly into the LLM's context window.
- **3. Generate:** LLM reads the explicit facts and answers accurately.



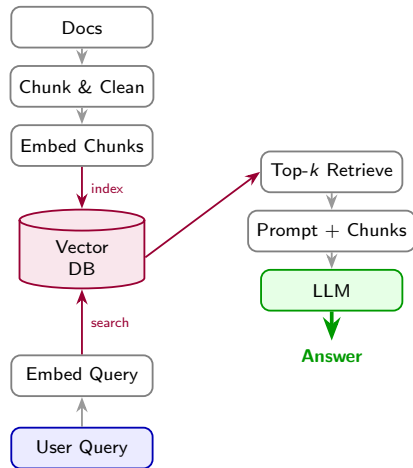
Baseline RAG: The Standard Pipeline

Two stages

- **Offline indexing:** Documents → chunking → embeddings → Vector DB.
- **Online answering:** Query → embedding → top- k retrieval (optionally rerank) → prompt stuffing → LLM.

What makes it "baseline"?

- Similarity search over independent chunks (no explicit global structure).
- Answer quality depends on chunking, embeddings, k , and prompt template.



The Baseline Vector RAG Bottleneck

Engagement Question

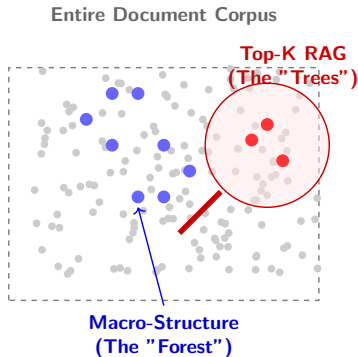
- *If we ask, "What is the main theme connecting all Q3 reports?", why will standard RAG fail?*

The "Global Sensemaking" Failure

- *Standard RAG:* Embeds query, retrieves top-K chunks.
- Designed purely for specific fact retrieval.

Missing the Forest for the Trees

- "Overall themes" require full dataset synthesis.
- RAG drops 99% of documents; LLM remains blind to macro-structures.



GraphRAG Phase I: LLM Extraction

Converting Text to Topology

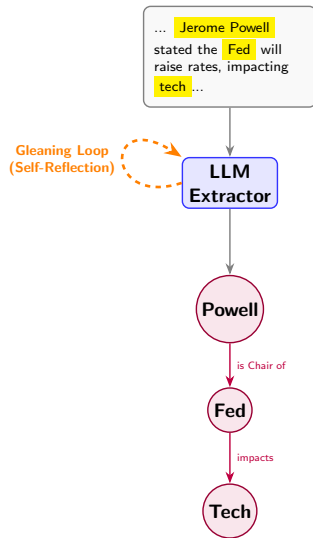
- We must expose the implicit graph hidden in unstructured text.

The Extractor LLM

- Documents are chunked and fed to an LLM.
- The LLM actively extracts **Entities** (Nodes) and **Relationships** (Edges).

The "Gleaning" Loop

- LLMs are lazy and miss entities on the first pass.
- *Solution:* We force the LLM to self-reflect and search the text again, densifying the graph.



GraphRAG Phase II: Topological Clustering

The Graph Context Problem

- We now have a massive Knowledge Graph.
- It still will not fit in the LLM's context window!

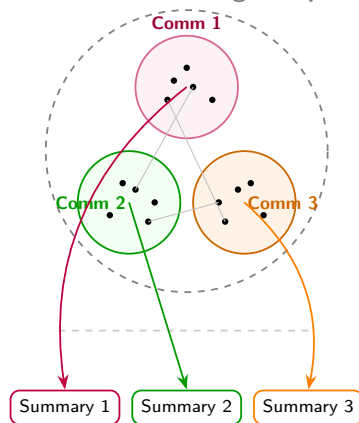
The Leiden Algorithm

- We apply classic graph theory to partition the network into dense subgraphs (**Communities**).

Hierarchical Summarization

- We prompt the LLM to read the entities in each community and write a **short text summary**.
- *Result:* Millions of tokens compressed into high-level semantic themes!

Massive Knowledge Graph



GraphRAG Phase III: Map-Reduce Querying

Executing Global Sensemaking

- Query: "What are the major themes?"

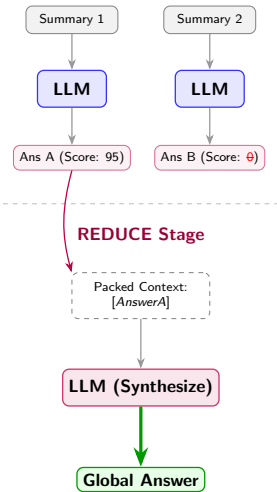
The MAP Stage (Parallel)

- Prompt the LLM with the query + *one* summary at a time.
- LLM generates an answer and explicitly **Scores it (0-100)**.

The REDUCE Stage

- Discard all answers scoring 0 (Topological noise).
- Pack top answers into the context window for the LLM to synthesize the **Global Answer**.

MAP Stage (Parallel)



Week 6 Conclusion: Overcoming the Bottleneck

The Universal Sequence Learner

- Transformers equate to dense Graph Attention Networks (Data-induced priors).

Constraining the Prior

- Bridged 1D → N-D using TabPFN (In-Context) and Graphormer (Injections).
- Locked outputs using FSM Logit Masking.

The Neuro-Symbolic Synthesis

- Text-to-SQL and GraphRAG achieve compositional “Neuro-Symbolic” integration.
- *Next Week*: Opening the black box via Mechanistic Interpretability.

